Software Requirements Specification (SRS)

Smart Traffic Complaint and Monitoring System

Project team: Arya, Ansh, Arnav, Avyukt

Technology baseline: MERN (MongoDB, Express.js, React, Node.js)

Document status: Prepared for submission review; test execution results must be filled only after tests are run.

1. Introduction

1.1 Purpose
This document defines the functional, non-functional, and security requirements for a web-based system that lets citizens submit traffic-related complaints and lets authorized administrators review, prioritize, assign, and update them.

1.2 Scope
The system supports account registration and login, complaint submission, viewing and tracking a citizen's own complaints, and administrator workflows for reviewing and managing submitted complaints. The proposed implementation uses the MERN stack. External traffic-authority integrations, automated violation detection, and live traffic-sensor integrations are outside the current scope.

1.3 Definitions
- FR: Functional Requirement; NFR: Non-Functional Requirement.
- Citizen: a registered user who submits and tracks complaints.
- Administrator: an authorized user who manages complaints.
- Complaint status: Submitted, Under Review, In Progress, Resolved, or Rejected.

1.4 Intended users
Citizens, system administrators, developers, testers, and project evaluators.

2. Overall Description

2.1 Product perspective
The application is a browser-based client-server system. React provides the user interface, Node.js and Express.js provide REST endpoints and business rules, and MongoDB stores user and complaint documents.

2.2 Assumptions and constraints
- Users have a modern web browser and network connectivity.
- Administrative accounts are provisioned or granted by an authorized project administrator.
- The initial release supports complaint text, category, and location description; photo uploads and map integrations are not required for the core release.
- The application must enforce authorization on the server, not only by hiding UI controls.

3. Functional Requirements

ID	Requirement
FR-01	The system shall allow a visitor to register using a name, unique email address, and password.
FR-02	The system shall allow a registered user to log in and log out.
FR-03	The system shall allow an authenticated citizen to submit a complaint with category, description, and location description.
FR-04	The system shall assign a unique complaint identifier and creation timestamp to each successfully saved complaint.
FR-05	The system shall display a confirmation containing the complaint identifier after successful submission.
FR-06	The system shall allow a citizen to list only complaints associated with that citizen’s account.
FR-07	The system shall display the current status for each complaint visible to the citizen.
FR-08	The system shall allow an authorized administrator to list all complaints.
FR-09	The system shall allow an administrator to filter complaints by status and category.
FR-10	The system shall allow an administrator to set a complaint priority from Low, Medium, or High.
FR-11	The system shall allow an administrator to assign a complaint to a responsible staff member or leave it unassigned.
FR-12	The system shall allow an administrator to update status to Submitted, Under Review, In Progress, Resolved, or Rejected.
FR-13	The system shall display administrator dashboard counts for total complaints and counts by status.
FR-14	The system shall reject complaint submissions missing category, description, or location, and shall return field-level validation feedback.
FR-15	The system shall reject administrator-only API requests from users who do not have the administrator role.

4. Non-Functional Requirements

ID	Requirement
NFR-01 Performance	On the agreed test environment with up to 1,000 complaint records, at least 95% of standard complaint list/detail API requests shall complete within 3 seconds, excluding external services.
NFR-02 Usability	In an acceptance test, at least 4 of 5 first-time test users shall submit a valid complaint within 3 minutes after login, using the provided instructions.
NFR-03 Availability	During a 30-minute acceptance run, the application shall be reachable for at least 29 minutes, excluding announced planned restarts.
NFR-04 Compatibility	The application shall support the current stable versions of Chrome and Firefox at the time of testing.
NFR-05 Maintainability	The codebase shall separate React presentation, Express route/controller logic, business validation, and database access into identifiable modules.
NFR-06 Reliability	For invalid requests or failed database operations, the API shall not return a success response and shall return a documented error response.
NFR-07 Data integrity	Each saved complaint shall include a unique identifier, submitting user, category, description, location description, creation timestamp, and status.
NFR-08 Password protection	Passwords shall be stored as salted password hashes using a suitable password-hashing library; plaintext passwords shall not be stored.
NFR-09 Authorization	Every protected API endpoint shall verify authentication and enforce role and ownership checks on the server.
NFR-10 Input validation	The API shall validate required fields, accepted status/priority values, and reasonable input lengths before database writes.

5. Security Requirements

5.1 Security objectives

- SO-01 Confidentiality: prevent users from viewing complaint records they are not authorized to access.
- SO-02 Integrity: prevent unauthorized creation or modification of complaint data and statuses.
- SO-03 Accountability: associate each complaint and administrative change with an authenticated account.

5.2 Security requirements

ID	Security requirement
SR-01	The system shall require authentication before a user can submit or view private complaints.
SR-02	The server shall enforce administrator role checks for listing all complaints, changing priority, assigning complaints, and updating status.
SR-03	The system shall store passwords only as salted password hashes and shall never return password hashes in API responses.
SR-04	The server shall validate and constrain user-provided values before processing them.
SR-05	Database operations shall use the MongoDB driver or ODM safely and shall not treat user input as executable query structure.
SR-06	Error responses shall not expose stack traces, secrets, password data, or internal database details.

6. Use-Case Diagram

Figure 1 shows the principal citizen and administrator interactions for the core release.

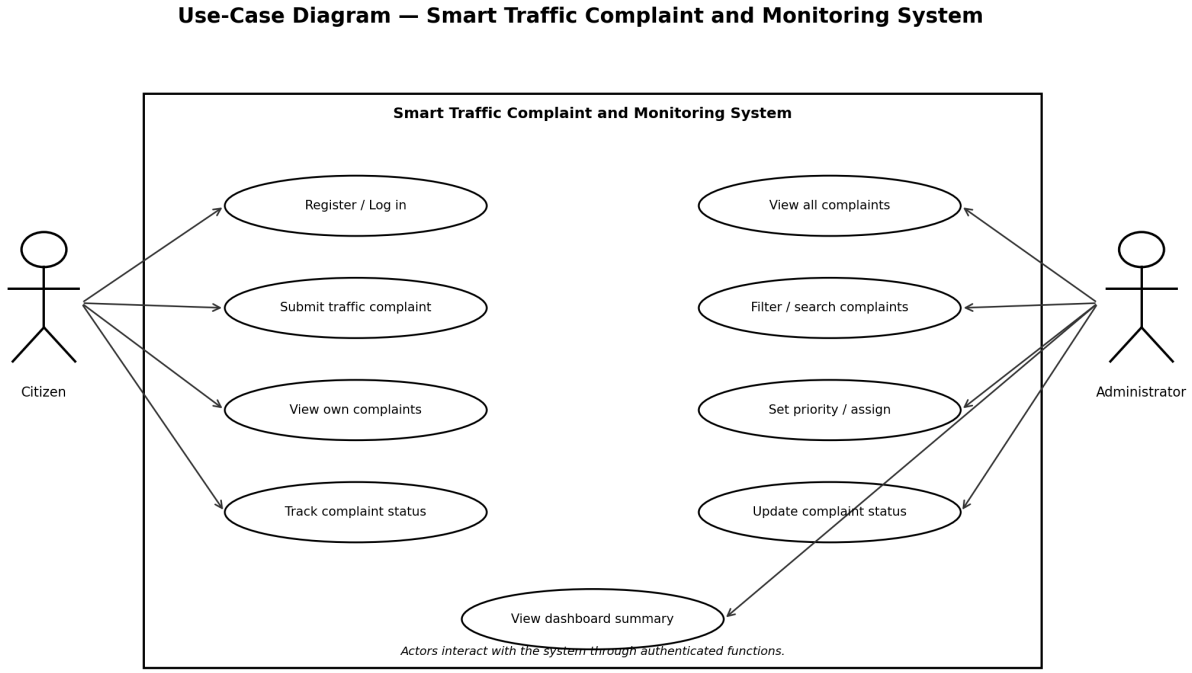


Figure 1. Use-case diagram for the Smart Traffic Complaint and Monitoring System.

7. Key Use Cases

UC-01 Submit traffic complaint

- Primary actor: Citizen.

- Precondition: citizen is authenticated.
- Main flow: enter category, description, and location; submit; server validates; system saves complaint with unique ID, timestamp, and Submitted status; UI shows confirmation.
- Alternative flow: if required data is missing or invalid, system returns validation errors and does not save the complaint.
- Postcondition: a valid complaint is stored and associated with the submitting citizen.

UC-02 Update complaint status

- Primary actor: Administrator.
- Precondition: administrator is authenticated and has the administrator role.
- Main flow: open complaint; choose an allowed status; submit update; server validates role and status; database is updated; UI displays the new status.
- Alternative flow: unauthorized user receives a forbidden response; unknown complaint receives not-found response; invalid status is rejected.
- Postcondition: complaint status is updated only by an authorized administrator.

8. Acceptance Criteria

- A citizen can register, log in, submit a valid complaint, and view its identifier and status.
- A citizen cannot retrieve another citizen’s private complaint by changing an ID in a request.
- A non-administrator cannot use administrator endpoints.
- Invalid required fields are rejected without creating a complaint.
- Administrator status and priority values are restricted to the documented allowed values.